

Homework 1: Relational Algebra and SQL

COSC 3337 — Databases — Fall 2026

Released	Wednesday, September 9
Due	Sunday, September 20, 11:59 pm
Total	100 points, plus 5 bonus

Before you start

This assignment has two halves that use the same database. Part 0 is setup. Part 1 is written. Part 2 is code. The last three SQL questions ask you to translate expressions you wrote in Part 1, so **do Part 1 first**.

What to submit: In a zip file, include:

1. `hw1.pdf` — your answers to Part 0 and Part 1.
2. `hw1.sql` — your answers to Part 2, in the format described on page 10.

Typesetting bonus (5 points).

- **By hand** You may hand-write your answers to Part 1. To submit your assignment, use a scanning app to submit a PDF, not a photograph. Make sure the scan is clear, your handwriting is legible, and each problem is clearly labeled. Submit the PDF scan.
- **With $\text{\LaTeX}$ (5 point extra-credit)** If you typeset Part 1 in $\text{\LaTeX}$, you earn 5 bonus points. A template with every symbol you need already defined as a macro is posted alongside this handout. I recommend using Overleaf to make a copy of the template and compile your answers. Export a compiled PDF to submit.

I recommend completing Part 0 by September 12 so we have time to resolve any issues.

Part 0 — Setting up your VM and code

(5 points)

0.0 Obtain your VM

Each person will have access to their own personal virtual machine. **Email me to get your personal VM address and temporary password. Use the subject line: COSC3337 VM Request.** I will assign you a VM to use for the rest of the semester. It has MySQL already installed and you will have full admin access to it.

Read this before you type anything

You are the root user on your VM. You have complete control of the machine and there are no guardrails: every command you type is permitted, including the ones that destroy it. These VMs are built from a public image, and machines like them are scanned and attacked continuously. There is also **no backup image and no console access**: if you lock yourself out or damage the operating system, the only repair available is deleting your VM and creating a new one, which you must email me to do.

The VM is replaceable, your work is not. Make sure to backup your work as you go.

0.1 Connect and change your password

Connect at the address you were emailed. You log in as root:

```
| ssh root@your-vm-address
```

Do not skip this step

The image ships with a password that is not secret, so change it now:

```
| passwd
```

Open a second terminal and log in with your new password before you close the first one. If the new password did not take, or you entered something other than what you thought, the still-open session is the only thing standing between you and an inaccessible VM. Test first, close second. Store the new password in a password manager. There is no recovery path.

0.2 Load the data

Clone the course repository onto your VM and run the setup script:

```
| git clone https://github.com/isabelcachola/COSC3337-FA26-homeworks
| cd COSC3337-FA26-homeworks/HW1
| bash setup_db.sh
```

‘setup_db.sh’ builds both databases and confirms you can reach them. It is safe to run again at any time. It restores both databases to a clean state, so it is also your repair command if you damage your database.

From then on, connect with:

```
| mysql soundwave
```

If something breaks

Re-run `bash setup_db.sh`. That restores both databases to a clean state and fixes any damage to the data.

If MySQL itself will not start, or you cannot log in at all, email me or come to Office Hours. Replacement VMs take up to one business day.

0.3 Verify

(5 points)

Run these two commands to verify your setup.

First, your account and the date your password was last changed:

```
| echo "==" $(whoami)@$(hostname) =="  
| passwd -S $(whoami)
```

Then the database check. It is one command even though it takes four lines. Copy all four:

```
| mysql -t -e "SELECT VERSION();  
| SELECT 'artist' AS t, COUNT(*) AS n FROM soundwave.artist  
| UNION ALL SELECT 'album', COUNT(*) FROM soundwave.album  
| UNION ALL SELECT 'track', COUNT(*) FROM soundwave.track;"
```

The above commands prove your account works, your password has been changed (the date in the `passwd -S` output), your MySQL access is set up, and your data loaded.

0.4 Back up your work

It is vital you back up your work as you complete this assignment. I will be using GitHub in this class to share starting code for homeworks, and you will use it to back up your code. An introduction to git is available here: <https://www.geeksforgeeks.org/git/git-introduction/>.

Step 1 — Make yourself a repository. On github.com, create a new repository called `cosc3337-work`.

Settings when you create the repo

Choose Private. It will contain your homework answers, and a public copy lets anybody find and copy them.

Leave it completely empty. Do *not* tick “Add a README”, a `.gitignore`, or a license. Those create a commit, and your first push below will be rejected because your history and GitHub’s would have nothing in common.

Step 2 — Give GitHub an SSH key. GitHub needs to recognise your VM before it will let you push. Create a key and print it:

```
ssh-keygen -t ed25519 -C "you@email.com"
cat $HOME/.ssh/id_ed25519.pub
```

Press Enter at all three prompts to accept the default location and leave the passphrase empty. The cat prints one long line beginning ssh-ed25519 and ending with your email address — copy the whole thing.

On github.com go to Settings → SSH and GPG keys → New SSH key, paste it in, title it something like COSC 3337 VM, and click Add SSH key. Then check it worked:

```
ssh -T git@github.com
```

The first time, it asks whether to trust GitHub’s fingerprint — type *yes*. You should then see *“Hi YOUR-USERNAME! You’ve successfully authenticated, but GitHub does not provide shell access.”* That message is success means you’ve set up your repo correctly.

If you ever get a replacement VM

The private half of this key lives on the VM and cannot be recovered from anywhere else. If your VM is replaced, delete the old key on GitHub (Settings → SSH and GPG keys) and generate a fresh one, rather than leaving a key GitHub trusts on a machine you no longer control.

Step 3 — Point your VM at your repository. Inside your cloned COSC3337-FA26-homeworks directory:

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
git remote rename origin upstream
git remote add origin git@github.com:YOUR-USERNAME/cosc3337-work.git
git push -u origin HEAD
```

Note the shape of that origin URL. The SSH form is `git@github.com:` followed by your username and repository, with a **colon** after github.com, not a slash. A slash there is the most common reason this step fails.

When you clone the repo, it points to my repository. Renaming it to upstream keeps that link for receiving updates for future homeworks, and origin now points at yours for saving your work. HEAD means “whatever branch I am on,” and -u remembers the choice, so from now on git push pushes to your current branch.

upstream keeps the `https://` address it was cloned with. That is deliberate: my repository is public, so pulling from it needs no authentication at all. Only pushing to *your* repository needs the key.

Note: These instructions use the SSH protocol; details are here: <https://docs.github.com/en/authentication/connecting-to-github-with-ssh/about-ssh>. If you would rather use HTTPS, you may. Use the `https://` form of the URL for origin and authenticate with

a *personal access token* generated under Settings → Credentials → Personal access tokens, pasting the token where git asks for a password. With HTTPS you will also want `git config --global credential.helper "cache --timeout=36000"`, so that you type the token about once per session rather than on every push. See <https://docs.github.com/en/get-started/git-basics/set-up-git#connecting-over-https-recommended> for more details about connection protocols.

Step 4 — Add me as a collaborator. On your repository's GitHub page: Settings → Collaborators → Add people, and invite `isabelcachola`. I will accept.

This is how I can look at your work if something goes wrong, recover a file you lost with your VM, or see what you had at the deadline if a submission goes astray. **Do it now**, while you are already on the page. A private backup I cannot reach is not much use to either of us on the day you need it.

Then, every time you stop working:

```
git add hw1.sql
git commit -m "<what you did this session>"
git push
```

And when I post a correction or the next homework, collect it with:

```
git pull upstream main
```

That is what upstream was for. Because your repository began as a clone of mine, the two histories share a common ancestor and this merges cleanly.

If GitHub fights you, do not lose the assignment over it. Copy the file down to your own computer instead, and sort out git afterwards:

```
scp root@your-vm-address:COSC3337-FA26-homeworks/HW1/hw1.sql .
```

Step 5 – Committing your first file

Run the following commands:

```
echo "==" $(whoami)@$(hostname) ==" &>> setup-outputs.txt
passwd -S $(whoami) &>> setup-outputs.txt

mysql -t -e "SELECT VERSION();
SELECT 'artist' AS t, COUNT(*) AS n FROM soundwave.artist
UNION ALL SELECT 'album', COUNT(*) FROM soundwave.album
UNION ALL SELECT 'track', COUNT(*) FROM soundwave.track;" &>> setup-outputs.txt
```

These commands run all of the setup verification commands from above and append them to the file `setup-outputs.txt`. The contents of `setup-outputs.txt` should look like this:

```
== root@{hostname} ==
root P 2026-09-02 0 14600 14 -1
+-----+
| VERSION() |
+-----+
| 8.0.46-0ubuntu0.24.04.4 |
+-----+
+-----+-----+
| t      | n      |
+-----+-----+
| artist | 200 |
| album  | 687 |
| track  | 6660 |
+-----+-----+
```

How to get full credit for Part 0

Commit setup-outputs.txt and push it to your repo. The 5 points associated with Part 0 will be awarded if:

1. I can view your repo and it is Private.
2. The contents of setup-outputs.txt show that you correctly setup your VM, including changing the password and setting up the database.

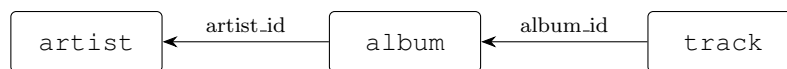
Part 1 — Relational algebra

(50 points)

Work these out by hand against the instance printed below. Do not use MySQL to find the answers; the point is to be able to reason about an expression without running it. (You will get to check three of them in Part 2.)

The schema

```
artist(artist_id, name, country, formed_year)
album(album_id, title, artist_id, release_year, genre)
track(track_id, album_id, title, duration_sec)
```



Arrows run from each foreign key to the primary key it references. One thing worth remembering: An artist reaches a track only through $\text{artist} \rightarrow \text{album} \rightarrow \text{track}$, so any question about both ends needs all three relations.

The instance

artist

artist_id	name	country	formed_year
A1	Neon Vale	USA	2011
A2	Sofia Marchetti	Italy	2004
A3	The Tidal	UK	2016
A4	Kwame Osei	UK	2013
A5	Hollow Pines	USA	2009
A6	Yuki Tanabe	Japan	2018
A7	Marta Reyes	Spain	2015

album

album_id	title	artist_id	release_year	genre
B1	Glass Harbor	A1	2019	Indie
B2	Second Light	A1	2022	Indie
B3	Notturmo	A2	2018	Classical
B4	Tide and Timber	A3	2021	Rock
B5	Accra Nights	A4	2020	Afrobeat
B6	Dust Chorus	A5	2017	Folk
B7	Dust Chorus II	A5	2022	Folk
B8	Kirameki	A6	2023	Electronic
B9	Cielo Bajo	A7	2021	Latin

track			
track_id	album_id	title	duration_sec
T01	B1	Harbor Lights	245
T02	B1	Saltwater	198
T03	B1	Glass	312
T04	B2	Second Light	224
T05	B2	Undertow	187
T06	B3	Notturmo I	402
T07	B3	Notturmo II	355
T08	B4	Timber	231
T09	B4	Spring Tide	265
T10	B5	Accra Nights	208
T11	B5	Highlife	243
T12	B6	Dust	279
T13	B7	Chorus	196
T14	B8	Kirameki	182
T15	B9	Cielo	254
T16	B9	Bajo	219

Notation

Use the operators class: σ (selection), Π (projection), $\times$ (Cartesian product), $\cup$, $\cap$, $-$, and γ (aggregation).

A subscript belongs to its operator, not to the whole expression: the predicate sits under the σ , and Π , ρ and γ carry their attribute list, new name and aggregate the same way.

RA1. (7 points)

Find the titles of all tracks shorter than 200 seconds. Write the expression, then list the resulting tuples.

RA2. (7 points)

Write an expression for the distinct countries that appear in **artist**.

Then answer, in one or two sentences: how many tuples does your result contain, how many rows does **artist** have, and why do the two numbers differ?

RA3. (7 points)

Find the names of artists who have at least one track shorter than 200 seconds. Write the expression, then list the resulting tuples.

RA4. (7 points)

Find the names of artists who have never released an album in the Indie genre. Write the expression, then list the resulting tuples.

RA5. (7 points)

For each album, find the total duration of all tracks on that album. Write the expression, then list the resulting tuples.

RA6. (8 points)

A classmate wrote the following, intending to find each Folk album together with the name of the artist who released it:

$$\Pi_{\text{artist.name, album.title}} (\sigma_{\text{album.genre}=\text{"Folk"}} (\text{artist} \times \text{album}))$$

- Exactly how many tuples does this expression produce against the instance above? Show how you got the number.
- Explain in one or two sentences what is wrong with it.
- Write a correct expression, and list the tuples it produces.

RA7. (7 points)

For each pair below, say whether the two expressions are equivalent — that is, whether they produce the same result on *every* possible instance of the schema, not just this one. Justify each answer in a sentence.

(a)

$$E_1 = \Pi_{\text{title}} (\sigma_{\text{release_year} \geq 2020} (\text{album})) \quad E_2 = \sigma_{\text{release_year} \geq 2020} (\Pi_{\text{title}} (\text{album}))$$

(b)

$$E_3 = \sigma_{\text{duration_sec} > 240} (\sigma_{\text{album_id}=\text{"B1"}} (\text{track})) \quad E_4 = \sigma_{\text{album_id}=\text{"B1"}} (\sigma_{\text{duration_sec} > 240} (\text{track}))$$

If they are equivalent, state the general rule your answer relies on.

Part 2 — SQL

(45 points)

Now the same schema, in MySQL, on your VM.

Q1–Q5 run against the soundwave database, which holds a few hundred artists and several thousand tracks. **Q6–Q8 run against soundwave_small**, the instance included in Part 1. Those three are translations of your own Part 1 answers, and a correct translation should return the tuples you worked out by hand.

Submission format

Put all 8 answers in one file called `hw1.sql`, like this:

```
-- Q1
SELECT ...;

-- Q2
SELECT ...;
```

The rules are strict because the file is graded by a script:

- Each answer is preceded by `-- Qn` alone on its own line.
- Each answer is **exactly one statement**, ending in a semicolon.
- Answers appear in order, Q1 through Q8. Do not skip numbers — if you are not answering one, leave the marker with no statement under it.
- Any other comments you want to write are fine anywhere else.

Check your file before you submit

Run the posted validator against your file:

```
| python3 validate_submission.py hw1.sql --check-runs
```

It tells you whether your file parses into 8 queries and whether each one runs. You can run the script without the `--check-runs` flag to only check the formatting. It does *not* tell you whether your answers are right. A file that fails the validator will lose points.

How Part 2 is graded

Your query's result is compared against the reference result. So:

- **Return exactly the columns asked for, in the order given.** Column names matter only where the question names them.
- **Row order matters only where the question says to order.** Where it does, your `ORDER BY` must match.

- **Duplicate rows are compared.** If the reference returns a value once and you return it three times, that is wrong.
- Every question can be answered with a single `SELECT`, and none of them should modify anything. If a query of yours changes the database, reload it with `bash setup_db.sh` before carrying on, or your later answers will be run against damaged data.

Questions Q1–Q5 (database: **soundwave**)

Q1. *(5 points)*

List each distinct country appearing in `artist`, excluding artists whose country is not recorded. Order alphabetically. One column, named `country`.

Q2. *(5 points)*

In a single row, report how many rows `artist` has and how many artists have a recorded country. Two columns, named `total_rows` and `known_countries`.

Q3. *(6 points)*

List the `album_id` and `title` of every album whose release year is not recorded, ordered by `album_id`.

Q4. *(6 points)*

List the `artist_id` and `name` of every artist who has not released any album, ordered by `artist_id`.

Q5. *(6 points)*

For each genre, list the genre and the average duration of all tracks on albums of that genre, rounded to one decimal place. Order by genre. Two columns, named `genre` and `avg_duration`.

Questions Q6–Q8 (database: `soundwave_small`)

Q6. *(5 points)*

Translate your RA1 expression into SQL. One column, named `title`, ordered by title. Your result should be exactly the tuples you listed in RA1.

Q7. *(6 points)*

Translate your RA3 expression into SQL. One column, named `name`, ordered by name.

Count your rows and compare with RA3. If the two disagree, do not just add something until they match. Work out what Π does that `SELECT` does not, and then fix it.

Q8. *(6 points)*

Translate your RA4 expression into SQL. One column, named `name`, ordered by name.

Total: 100 points (Part 0: 5, Part 1: 50, Part 2: 45), plus 5 bonus for a $\text{\LaTeX}$ -typeset Part 1.